

Setup in six lines. The task is two-number arithmetic posed in words (What is the sum of 621 and 5068?, answered 0-shot); the same problems can be posed in symbols ($621 + 5068 =$), the “symbol surface”. The model is TinyStories-1B (16 layers, 32 heads per layer, width 2048), pretrained on children’s stories with no arithmetic. Five parents are fine-tuned on the word task: the *elicit parent* (TS1B-latent: the base plus symbol arithmetic plus a word $\leftrightarrow$ symbol binding, so the capability is present but never shown on the task); the *symbol engine* (the base plus symbol arithmetic only, no binding); the *format-installed* parent (the base taught only to emit a number in the right format, no arithmetic); the *blank* base; and Llama-3.2-1B, an off-the-shelf second elicit parent used in the parent-only metrics. *Elicit* = fine-tune the elicit parent; *teach* = fine-tune the format-installed parent (the blank base is the control). Two methods: full FT updates every weight; LoRA trains rank-512 adapters on frozen weights. Metrics marked ★ need only the parent and are measured on all five parents (Table 10).

endpoint	parent	method	data	steps (passes)	stop	exact match
elicit FT	TS1B-latent	full FT	4M	21,500 (0.7)	converged	0.986
teach FT	format-installed	full FT	4M	58,500 (1.9)	converged	0.424
elicit LoRA (sweep, snapshots)	TS1B-latent	LoRA $r=512$	1M	11,500 (1.5)	converged	0.981
teach LoRA (sweep)	format-installed	LoRA $r=512$	1M	–	converged	0.079
teach LoRA (snapshots)	blank	LoRA $r=512$	1M	25,000 (3.2)	converged	0.093
teach LoRA 4M (control)	blank	LoRA $r=512$	4M	39,500 (1.3)	converged	0.139
teach FT from the blank base (control)	blank	full FT	4M	72,000 (2.3)	converged	0.819

Table 1: The main pair (top) and the endpoints used where noted (bottom). LoRA learning rate 3.53×10^{-4} , full FT 2×10^{-5} ; batch 128; seed 316; training stops when validation loss stops improving. Exact match = 0-shot accuracy on 1,024 held-out problems. The 4M file is the 1M file plus 3M new unique problems, disjoint from every evaluation set.

M1 Circuit overlap — the same circuit vs. a new circuit

Elicit vs. teach, in one sentence. Does the fine-tuned model solve the task with the circuit its parent already had (elicit) or with a circuit that did not exist before (teach)?

How it is measured.

- *Nodes.* The model is cut into 528 parts: 512 attention heads and 16 MLP blocks.
- *Scoring.* For each node, how much the answer’s logit would move if that node’s activation were swapped for its value on a different problem (first-order estimate: activation difference $\times$ gradient), averaged over 128–256 problem pairs. The top 32 are the circuit.
- *Comparing.* Jaccard@32 = shared nodes $\div$ union. Read against two references: the *ceiling* (split-half), the overlap of the same model’s circuit built from two halves of the data (how repeatable a circuit is at all), and the *type-matched chance*, what two unrelated circuits with the same MLP/head mix share (0.20–0.27 here).
- *Heads only.* The same overlap over the 512 heads alone, where circuits can actually differ (chance 0.03).
- *Functional check.* Overlap counts names; this counts effect. Corrupt a set of 32 heads (write in their activations from a different problem) and read how much of the model’s answer preference is destroyed, against random sets of 32 heads. A set identified on the *parent* and evaluated in the *child* tests reuse without depending on which names rank highest.

The numbers. Table 2.

- **Elicit:** parent $\leftrightarrow$ child 0.455 (chance 0.23, ceilings 0.60 / 0.56); heads only 0.333 against the parent’s own ceiling of 0.42. **Teach:** the parent has no circuit for the task (its node-score map is noise, M10), so the child’s is new; heads only 0.016.
- Functional check: the parent’s 32 heads, found before any target training, carry 0.73 of the elicited child’s behaviour (random heads 0.04; the child’s own top heads 0.88). The taught child’s own top heads carry only 0.46 of its behaviour, and 0.27 of the elicited child’s: teaching layers diffuse attention on the shared number-reading heads (M2) instead of inheriting a compact head circuit. Set overlap between the two children is at chance (0.306; heads only 0.103).

Comparison	Jaccard@32	type-matched chance	ceilings (A / B)	heads-only J@32
elicit parent $\leftrightarrow$ elicited child	0.455	0.23	0.600 / 0.561	0.333 (ceiling 0.42)
teach parent $\leftrightarrow$ taught child	undefined (parent map is noise; 0.231)	0.25	– / 0.524	0.016
elicit parent on the task $\leftrightarrow$ on symbols	0.524	0.27	0.600 / 0.306	0.280
elicit parent on symbols $\leftrightarrow$ elicited child	0.391	0.25	0.306 / 0.561	0.255
elicited child, full FT $\leftrightarrow$ LoRA	0.641	0.20	0.561 / 0.684	0.455
elicited child $\leftrightarrow$ taught child	0.306	0.25	0.561 / 0.524	0.103
taught child $\leftrightarrow$ taught child from the blank base	0.362	0.27	0.524 / 0.422	0.208

Table 2: Circuit overlaps of the full-FT children and their parents, mapped on the word task (0-shot). Uniform chance is 0.031 everywhere; ceilings are each map’s own split-half repeatability; the teach parent’s map fails the performing check (logit-diff -1.5 nats, M11), so its overlap is noise. Bottom: the elicit parent’s task circuit is its symbol-arithmetic circuit; the elicited circuit is method-independent; the two children against each other; two taught circuits against each other.

M2 Head roles — *the same reader heads vs. rediscovered ones*

Elicit vs. teach, in one sentence. Does the fine-tuned model keep the parent’s heads that read the two numbers (elicit) or have to find reader heads of its own (teach)?

How it is measured.

- Take a problem and a twin that differs in one thing only (a different first number).
- Learn the smallest set of attention heads such that copying their activations from the twin makes the model answer the twin’s problem (Desiderata Component Masking, Prakash et al. 2024). Those heads carry that piece of information.
- A set counts only if it flips the answer as often as copying every head does (the ceiling). Sets are compared across models by Jaccard (chance ≈ 0.05).

The numbers. Table 3.

- **Elicit:** the child’s number-reading heads are the parent’s, 0.69 / 0.86 (first / second number; reference sets = the elicit parent on symbols). **Teach** (against the same reference, since the teach parent has no reader heads, M14): the child finds a larger set that still contains most of the same heads, 0.88 / 0.66 — a graded difference, not an absence.
- A taught model that never learned the task has no compact set at all.

M3 Circuit formation time — *present at step 1 vs. assembled mid-run*

Elicit vs. teach, in one sentence. Is the final circuit present from the first step (elicit) or assembled somewhere in the middle of training (teach)?

How it is measured.

Comparison (first operand / second operand)	$ A $	$ B $	Jaccard	flip acc. / ceiling of A
elicited child (FT) $\leftrightarrow$ elicit parent on symbols	31 / 25	30 / 27	0.694 / 0.857	0.98 / 0.98 (at ceiling)
taught child (FT) $\leftrightarrow$ elicit parent on symbols	32 / 41	30 / 27	0.879 / 0.659	0.59 / 0.54 (0.59 / 0.55)
taught child (FT) $\leftrightarrow$ elicited child (LoRA)	32 / 41	36 / 30	0.659 / 0.690	–
elicit parent on symbols $\leftrightarrow$ elicited child (LoRA), symbol surface	30 / 27	32 / 30	0.879 / 0.900	at ceiling
elicit parent on symbols $\leftrightarrow$ elicited child (LoRA), operation role	25	24	0.750	at ceiling
elicited child (FT) $\leftrightarrow$ elicited child (LoRA)	31 / 25	36 / 30	0.811 / 0.833	0.98 / 0.98
taught child that did not learn (LoRA 4M) $\leftrightarrow$ elicited child (LoRA)	12 / 21	36 / 30	0.043 / 0.041	0.01 / 0.03 (0.20 / 0.14)

Table 3: Head roles by Desiderata Component Masking. Reference role sets = the elicit parent on the symbol surface, where it already answers. $|A|$, $|B|$ = size of the two role sets. Bottom: the references (the LoRA-elicited child keeps the parent’s roles almost exactly; a taught model that never learned the task has no roles to compare).

- Snapshots were saved along the LoRA runs (128 per run); the full-FT runs have none, so this metric uses the two 1M LoRA runs (Table 1: elicit LoRA and teach LoRA from the blank base).
- At each snapshot, build the circuit as in M1 and take its overlap with the run’s final circuit. The circuit exists once the curve leaves chance; it is finished once it reaches the ceiling.

The numbers. Figure 1.

- **Elicit:** 0.49 overlap with the final circuit at step 1 ($16\times$ chance), locked from step 1.5K of 11.5K. **Teach:** at chance until step 1.3K, built over steps 1.3K–5.4K of 25K — the stretch where its loss on new examples drops fastest.

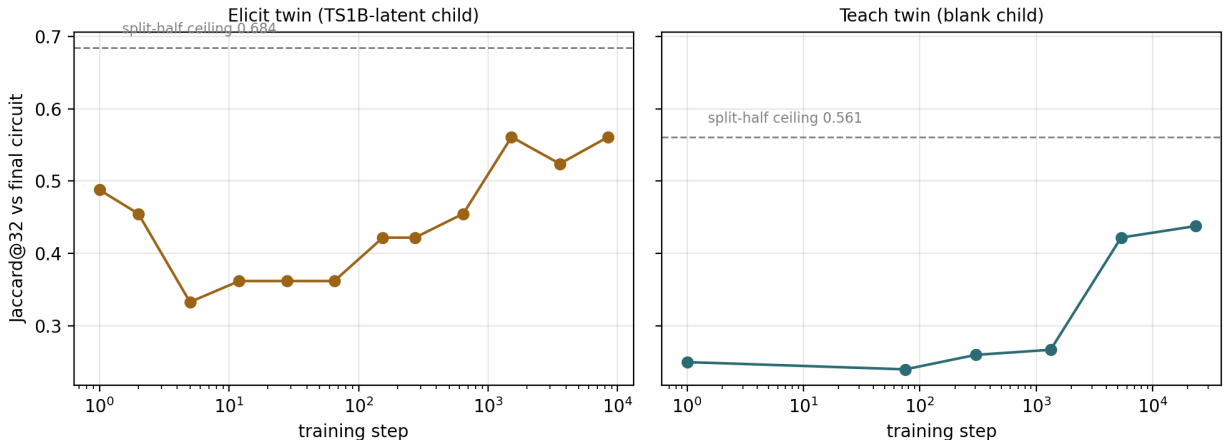


Figure 1: Circuit formation, elicit (left) vs teach (right); dashed = split-half ceiling.

M4 Gradient pressure — a dying gradient vs. a growing one

Elicit vs. teach, in one sentence. Does the gradient fade as training proceeds because there is nothing left to change (elicit), or grow because structure is still being built (teach)?

How it is measured.

- The size of the gradient (before clipping) at every training step.

- Compare its average over the first 1% of steps with the last 10%: a ratio above 1 means the gradient fades, below 1 means it grows. Also the total gradient accumulated over the run.
- Norms are comparable within one fine-tuning method only.

The numbers. Table 4, Figure 2.

- **Elicit:** fades $3.4\times$ from the first 1% to the last 10%. **Teach:** grows $1.4\times$, and accumulates $8.6\times$ the total gradient (378K vs 44K).
- Under LoRA the gap is larger ($40\times$ growth, $150\times$ the mass), aimed at attention weights — the number-reading heads of M2 being written. Figure 2 repeats both quantities across the sweep over training-set size n .

	elicit FT	teach FT	elicit LoRA 1M	teach LoRA 1M (blank)
steps	21,500	58,500	11,500	25,000
peak gradient norm (step)	34.0 (1)	70.9 (50,860)	0.441 (96)	17.4 (23,657)
mean norm: first 1% / middle / last 10%	4.80 / 1.85 / 1.39	2.69 / 4.86 / 3.77	0.183 / 0.060 / 0.056	0.169 / 5.21 / 6.80
first 1% $\div$ last 10%	$\times 3.4$ (decays)	$\times 0.7$ (grows $1.4\times$)	$\times 3.3$ (decays)	$\times 0.02$ (grows $\sim 40\times$)
cumulative gradient mass $\sum_t \ g_t\ $	44,021	378,170	783	116,877
gradient mass per step	2.05	6.46	0.068	4.68
mass share QK / VO / MLP	—	—	.18 / .55 / .27	.12 / .87 / .01

Table 4: Raw gradient strength (pre-clip global norm at every update). LoRA and full-FT norms are on different parameter sets and are compared within a method only; the LoRA pair is shown for the per-class split, which the full-FT trainer does not log.

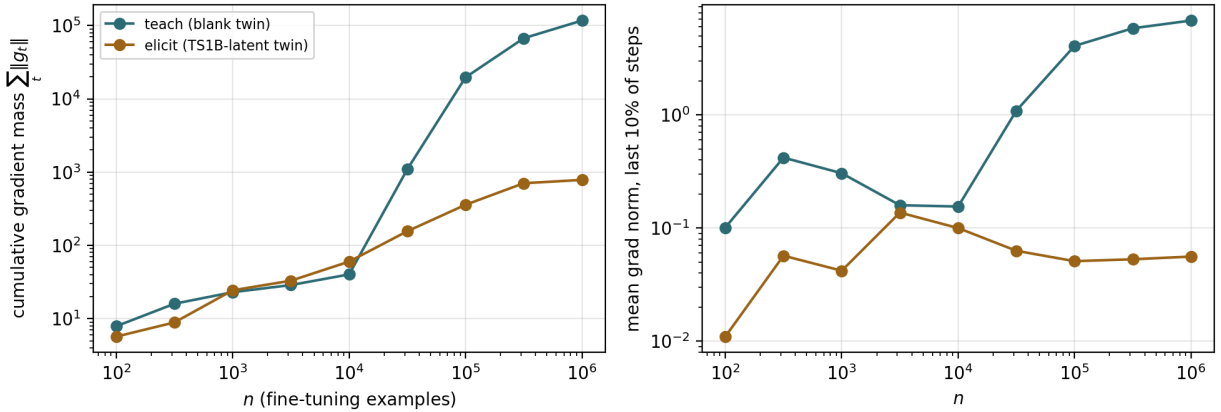


Figure 2: Cumulative gradient mass (left) and late-run gradient norm (right) vs n ; teal = teach, gold = elicit.

M5 Weight write — a small update vs. a large one

Elicit vs. teach, in one sentence. Is the change to the weights small and efficient (elicit) or large and expensive per unit of capability gained (teach)?

How it is measured.

- Relative weight change $\|\Delta W\|/\|W\|$ between child and parent, over all attention and MLP matrices.

- Along the LoRA runs (the only ones with snapshots): total distance the weights travel, and capability per unit travelled = final exact match $\div$ distance.

The numbers. Table 5.

- **Elicit** 0.050 vs **teach** 0.086: teaching writes $1.7\times$ more into the weights.
- Under LoRA: $3.8\times$ the distance for $40\times$ less capability per unit travelled.

	elicit	teach
relative write $\ \Delta W\ /\ W\$, full FT	0.050	0.086
relative write, LoRA 1M (teach from the blank base)	0.095	0.212
steps (LoRA 1M)	11,500	25,000
peak / final writing speed per step (LoRA 1M)	0.218 / 0.018	0.306 / 0.022
total weight travel $\ \Delta W\ _F$ (LoRA 1M)	120	457
final exact match (LoRA 1M)	0.981	0.093
capability per unit travel (LoRA 1M)	8.1×10^{-3}	2.0×10^{-4}

Table 5: Weight-space write. Top: the pair. Bottom: the LoRA runs, the only ones with snapshots for travel.

M6 What fine-tuning changes in the internal state — *a different change per problem vs. the same change every time*

Elicit vs. teach, in one sentence. When the child’s internal state is compared with the parent’s on the same problem, is the change different for every problem — the child has added that problem’s answer (elicit) — or is it mostly the same change regardless of the problem — the child has been moved into a new mode (teach)?

How it is measured.

- For each of 256 problems, take the model’s internal state at the moment it is about to answer (a 2048-number vector), in the parent and in the child, and subtract. That difference is “the change” fine-tuning made on that problem.
- Ask whether the 256 changes point the same way: the share of their total energy along the single best direction (*shared-direction score*; 1.0 = the same push every time).
- Control: recompute after removing the directions that spell the answer token, so “the same push” cannot simply be the answer.

The numbers. Figure 3, Table 6.

- **Elicit** 0.056 (each problem’s change is its own) vs **teach** 0.567 (over half of every change is one shared push), unchanged with the answer removed.
- The taught child from the blank base, which did have an output format to learn, scores 0.526 — no higher, so the shared push is the algorithm’s mode, not “output a number”.

M7 Latent reach ★ — *do the words already reach the machinery?*

Elicit vs. teach, in one sentence. Before any target training, does a natural-language question already put the arithmetic components into a problem-specific state (elicit), or is there nothing for the words to reach (teach)?

How it is measured.

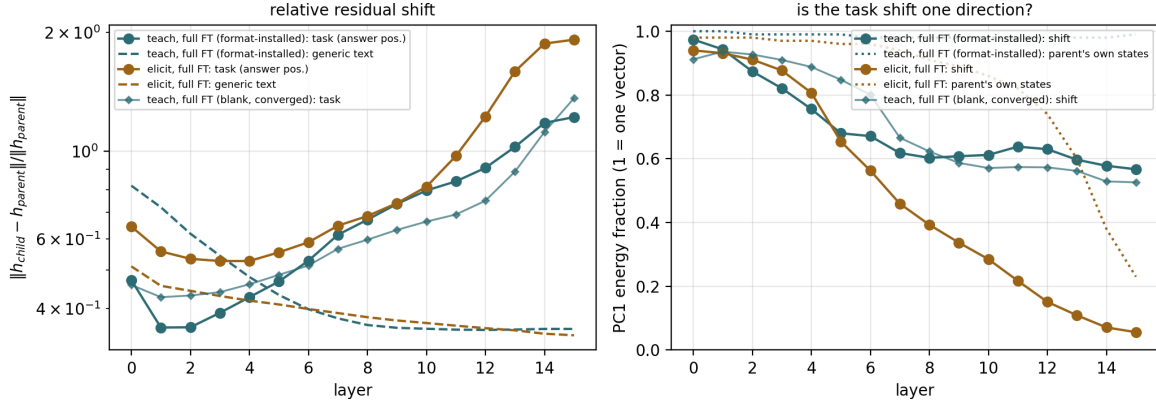


Figure 3: The pair (thick) with the blank-base taught control (thin diamonds). Left: relative change of the internal state by layer at the answer position (solid) and on generic story text (dashed). Right: shared-direction score (PC1) of the 256 per-problem changes by layer, with the parent’s own states dotted.

	elicit FT	teach FT	teach FT from the blank base (control)
final exact match	0.986	0.424	0.819
shared-direction score (PC1) of the change, last layer (parent’s own states)	0.056 (0.23)	0.567 (0.99)	0.526 (0.98)
same, after removing answer-token directions	0.061	0.561	0.528
cosine of each change to the mean change	0.23	0.80	0.74
size of the change relative to the parent’s state, last layer	1.92	1.22	1.36

Table 6: Change of the internal state parent→child at the answer position, 256 problems, fp32. Under LoRA (1M elicit / 4M teach from the blank base): score 0.063 vs 0.528, unchanged after removing the answer directions; the LoRA-taught size is 29× (an adapter artefact).

- Pose each problem twice, in words (**What is the sum of 621 and 5068?**) and in symbols ($621 + 5068 =$), and record each MLP block’s activation at the answer position.
- *Bridging index*: similarity between the two renderings of the same problem, minus the similarity between renderings of different problems. Positive = the words put the block into the state the symbols do.
- Second check: how alike the parent’s own states are across problems (shared-direction score; ≈ 1 = the same state for every problem).

The numbers. Table 6, “parent’s own states”.

- Bridging index of the arithmetic MLPs: ≈ 0 in the symbol engine (no binding), +0.07 to +0.13 (layers 7–15) in the elicit parent: the binding is what lets the words in.
- The elicit parent’s states are problem-specific (0.23); the teach parent sits in the same state for every problem (0.99; blank base 0.98).

M8 Answer depth ★ — *the answer already there vs. absent; finished one layer earlier vs. later*

Elicit vs. teach, in one sentence. Is the answer readable from the parent’s internal state before it can say it (elicit) or absent (teach); and after training, is it finished one layer earlier (elicit) or later (teach)?

How it is measured.

- Read the model’s intermediate state at each layer as if it were the output. Two lenses (a third, R, is in Table 7): the *logit lens* (unembed directly) and the *J-lens* (first transport the state to the final layer with the average Jacobian, then unembed — what the state is disposed to make the model say).
- Per layer: the answer’s median rank among 128K vocabulary entries (chance 64K) and the *logit-diff*, the preference for the correct answer over a mismatched problem’s (0 = no signal). Rank alone can be a number prior; the two are read together.
- *Settled depth*: the first layer from which the answer stays top-ranked.

The numbers. Figure 4, Table 7.

- **Before training:** the elicit parent ranks the answer 174 of 128K and prefers it by 2.1 nats while its actual output is a restatement of the question in symbols; the teach parent ranks it 452 with *logit-diff* 0.0 — a number, not this number.
- **After training:** the elicited child settles at layer 14, the taught child at layer 15.

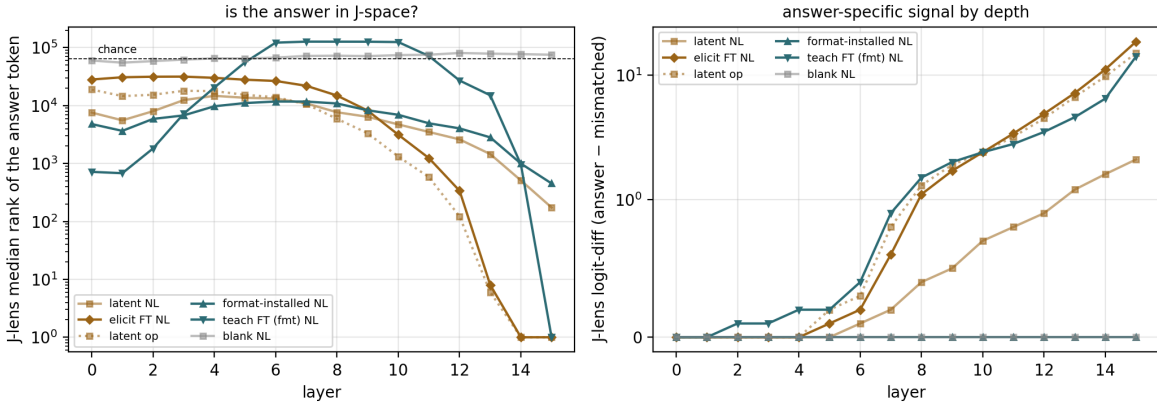


Figure 4: J-lens read-out of the answer token by layer: median rank (left, chance 64K) and logit-diff against a mismatched problem’s answer (right). Gold = elicit (parent: squares; child: diamonds; dotted: the parent on the symbol surface), teal = teach (parent: up-triangles; child: down-triangles), grey = blank base.

model	surface	own acc (first digit, 256 problems)	rank L12 (logit / J / R)	rank L14 (logit / J / R)	read-out	ld
elicit parent	NL	0.047	5,704 / 2,606 / 2,730	436 / 510 / 424	174	+2.1
teach parent (format-installed)	NL	0.004	8,394 / 4,036 / 5,417	1,106 / 997 / 998	452	0.0
elicited child (FT)	NL	0.988	1,797 / 338 / 492	1 / 1 / 1	1	+18.7
taught child (FT)	NL	0.535	23,909 / 27,044 / 25,529	381 / 961 / 1,848	1	+14.2
blank base	NL	0.000	76,049 / 80,840 / 78,243	77,707 / 77,320 / 79,116	75,441	0.0
taught child from the blank base (FT)	NL	0.844	18,988 / 4,123 / 9,620	170 / 138 / 459	1	+16.1
elicit parent	symbols	0.836	1,224 / 122 / 334	1 / 1 / 1	1	+15.1

Table 7: Median rank of the answer’s first digit token among 128K entries under the three lenses; “own acc” = the model’s own first-digit-token accuracy on the 256 problems; read-out = the last layer, where all lenses equal the model’s output; ld = logit-diff at the read-out (nats). Bottom: the blank base (rank without a number prior), the taught child from it (settles at layer 15 too), and the elicit parent’s own trajectory on symbols, which the elicited child reproduces.

M9 Switch-on by patching — *can the capability be turned on without training?*

Elicit vs. teach, in one sentence. With the parent’s weights frozen, does writing the fine-tuned child’s internal activations into the parent make it solve the task — possible only if the machinery is already there (elicit) and impossible if it is not (teach)?

How it is measured.

- Freeze the parent. On each problem, run the fine-tuned child, take its activations at the 32 circuit nodes of M1, and write them into the parent’s run at the answer position.
- Two variants: each problem gets the child’s activations on that problem (*per-prompt*), or every problem gets the child’s average activation (*mean vector*). Control: the same with 32 random nodes.
- Read exact match; “format” = share of outputs that are a number at all.

The numbers. Table 8, Figure 5.

- **Elicit:** the child’s per-prompt states switch the parent from 0.000 to 0.121 (random nodes 0.000, LoRA donor).
- **Teach:** the same patch into the teach parent gives 0.016.
- The mean vector changes only the output format: the patch that works is problem-specific.

Parent (patient) + child (donor), 32 circuit nodes	condition	format	exact match
elicit parent + elicited child’s per-prompt states	circuit nodes	0.387	0.121
teach parent + taught child’s per-prompt states	circuit nodes	0.836 (parent 1.000)	0.016
elicit parent + LoRA child’s per-prompt states	circuit nodes	0.359	0.125
	random-32 nodes	0.051	0.000
elicit parent + LoRA child’s mean vector	circuit nodes	0.164	0.000
blank base + blank-base taught child’s states	every condition	≤ 0.02	≤ 0.004

Table 8: Donor patching, weights frozen. The parents score 0.000 unpatched. Bottom: controls (random nodes, the mean vector, the LoRA child as donor, the blank-base pair).

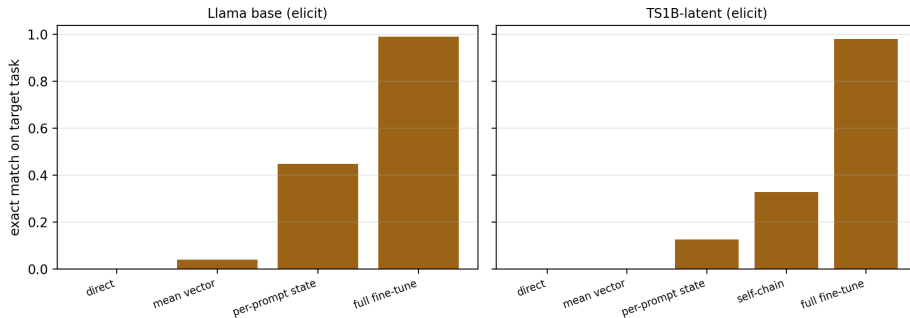


Figure 5: Donor patching on the elicit parent (right; left: the same on Llama-3.2-1B, the second elicit parent of M11–M15), weights frozen except the last bar: unpatched parent, mean vector, per-prompt states, and the fine-tuned child for scale.

M10 Repeatable task circuit in the parent ★ — a circuit for the task already there vs. noise

Elicit vs. teach, in one sentence. Before any target training, does the parent already have a stable set of components that raise the correct answer’s logit on the target task (elicit), or does its attribution map on the task have no structure at all (teach)?

How it is measured.

- Build the parent’s circuit on the target task as in M1, before any target training.
- *Is it defined?* The tool marks a map as noise when the parent’s answer logit-difference is below 1 nat.
- *Is it repeatable?* Build it twice from two halves of the problems and take the overlap.

The numbers. Table 9.

- **Elicit parent:** answers nothing, yet has a defined map (3.1 nats) that is as repeatable as a fine-tuned child’s, 0.600 (heads only 0.422). **Teach parent:** map flagged as noise (−1.5 nats).
- The elicit parent’s task circuit is its symbol-arithmetic circuit (0.524 overlap), the one the elicited child ends up running (M1).

parent, on the target task (0-shot)	exact match	answer logit-diff (nats)	map	split-half Jaccard@32 / 64 / 128
elicit parent	0.000	+3.1	performing	0.600 / 0.580 / 0.552
teach parent (format-installed)	0.000	−1.5	noise	—
blank base (16-shot)	0.000	−0.3	noise	—
elicit parent on the symbol surface	0.836	+11.0	performing	0.306 / 0.333 / 0.399

Table 9: The parent’s own task circuit. Chance overlap 0.031 / 0.065 / 0.138. The symbol surface map is less repeatable than the natural-language one despite a larger logit-difference: on symbols the parent already answers, and many components carry a little of the signal.

M11 Hidden preference ★ — *prefers the right answer without saying it vs. no preference*

Elicit vs. teach, in one sentence. Before any target training, does the parent already prefer the correct answer to a wrong one, even though it never produces it (elicit), or is it indifferent (teach)?

How it is measured.

- One forward pass per problem, 256 problems, on the parent alone.
- At the answer position, the logit of the correct answer’s first chunk minus the logit of a different problem’s answer. Mean in nats; 0 = no preference.

The numbers. Table 10.

- Elicit parent 2.31, Llama-3.2-1B 1.00, symbol engine 0.44, format-installed 0.06, blank 0.02: the order in which the five parents learn the task under fine-tuning (learning curves not shown here), from parents that produce none of the answers.

M12 Accelerating descent of the task loss ★ — *the drop speeds up along some direction vs. tapers along every direction*

Elicit vs. teach, in one sentence. Nudge the parent’s weights a little: is there a direction in which the task loss drops faster the further you go — a capability one push away (elicit) — or does the drop taper off whichever way you move (teach)?

How it is measured.

- Take the task loss at the parent’s weights and find the two ends of its curvature spectrum (Hessian-vector products, power iteration, 16 problems).

- The second derivative along a direction says what happens to the rate of descent: positive = the drop tapers, negative = it accelerates. $\lambda_{\max}$ and $\lambda_{\min}$ are the strongest taper and the strongest acceleration. *Negative share* = $-\lambda_{\min}/(\lambda_{\max} - \lambda_{\min})$: the fraction of that range on the accelerating side, 1 when every strong direction accelerates, 0 when none does. A ratio, so models of different loss scale compare.

The numbers. Table 10.

- Elicit parent 1.00, Llama-3.2-1B 0.64, symbol engine 0.48, format-installed 0.00, blank 0.00: again the order in which they learn. For an elicit parent there is a direction along which the loss falls faster and faster; for a teach parent every descent tapers off.

M13 Answer carried by the state ★ — *swapping the state moves the answer vs. nothing moves*

Elicit vs. teach, in one sentence. Does the parent’s state at the answer position already carry the answer causally — swap it between two problems and the preference follows (elicit) — or does swapping it change nothing (teach)?

How it is measured.

- Take two problems of equal length. Run the parent on the first while overwriting its state at the answer position, after layer L , with the state from the second (whole state, or a learned k -dimensional part of it; Distributed Alignment Search, Geiger et al. 2024).
- Count how often the parent now prefers the second problem’s answer. Controls: no swap, and a random subspace of the same size. 128 test pairs.

The numbers. Table 10.

- Swapping the whole state moves the preference by +0.39 (elicit parent), +0.26 (Llama), +0.30 (symbol engine) against +0.05 and +0.02 for the teach parents: every parent that can compute already holds the answer in its state.
- In the elicit parent a learned 64-dimensional part of the state recovers most of the effect (flip rate 0.64 vs 0.41 for a random 64-dimensional subspace).

M14 Number-reading heads ★ — *words already reach the arithmetic vs. they do not*

Elicit vs. teach, in one sentence. Do the parent’s attention heads already fetch the two numbers when the problem is posed in words (elicit), or not (teach) — and does having arithmetic suffice?

How it is measured.

- Head roles (M2) on the frozen parent with the word prompts, with one change: since the parent never answers, a head set counts when copying it from the twin problem moves the parent’s *preference* toward the twin’s answer.
- Report the size of the set and its flip rate against the copy-every-head ceiling.

The numbers. Table 10.

- Elicit parent 22 heads, Llama-3.2-1B 23; symbol engine, format-installed and blank: none. The engine has the arithmetic (M13; the numbers are present, M15) but no heads carrying them from the words to it — this is the metric that separates the two elicit parents from it.

M15 Operands present at the answer position ★ — *the numbers are there vs. weakly there*

Elicit vs. teach, in one sentence. Is the information the computation needs — the two numbers — already readable from the parent’s state at the moment it is about to answer?

How it is measured.

- Linear probe on the parent’s state at the answer position, about 2,000 addition problems, held-out split.
- Target: the two numbers, in the helix features language models use for numbers (Kantamneni & Tegmark 2025). Report the best layer’s R^2 .

The numbers. Table 10.

- Elicit parent 0.88, Llama-3.2-1B 0.86, symbol engine 0.90 vs format-installed 0.53, blank 0.49: parents that can compute have both numbers at the answer position.
- The answer itself is not linearly readable in any parent beyond the two numbers themselves; it is produced downstream, where the lens and the hidden preference find it.

	elicit		between	teach	
	elicit parent	Llama-3.2-1B	symbol engine	format-installed	blank
hidden preference (nats)	2.31	1.00	0.44	0.06	0.02
curvature: negative share	1.00	0.64	0.48	0.00	0.00
answer state swap: flip gain over no swap	+0.39	+0.26	+0.30	+0.05	+0.02
number-reading heads (first operand)	22	23	0	0	0
operands present (R^2)	0.88	0.86	0.90	0.53	0.49
repeatable task circuit (split-half)	0.600	—	—	noise	noise

Table 10: Parent-only metrics on the five parents. The first two order all five as their learning curves do; the next two split the parents that can compute from those that cannot; number-reading heads alone separate the two elicit parents from the symbol engine, which has the arithmetic but not the interface to the words.